



EXECUTIVE SUMMARY

6.2 Project #9: Setting Up Auditing Files

IT312

Cybersecurity Fundamentals

Promise Beshel

April 8th, 2026

Contents

1. Problem Statement	2
2. Introduction	2
3. Methodology: Steps Taken.....	2
3.1 Windows Server Configuration.....	3
3.2 Linux (Ubuntu) Auditing	7
4. Results Obtained	7
5. Challenges Encountered.....	9
6. Lessons Learned	10
7. Conclusion.....	10
8. Recommendations	10

List of Tables

Fig.1 Object Access and Logon Events Audit policy setup	3
Fig.2 Audit Credential Validation	3
Fig.3 Audit User Account Management and Audit Security Group Management	4
Fig.4 Audit Account Lockout, Audit Logoff, and Audit Special Logon	4
Fig.5 Audit File System, Audit Registry, and Audit Kernel Object	5
Fig.6 Audit Authentication Policy Change and Audit Authorization Policy Change	5
Fig.7 Audit IPsec Driver, Audit Security State Change, and Audit System Integrity	6
Fig.8 Audit IPsec Driver, Audit Security State Change, and Audit System Integrity	6
Fig.9 Five specific watches set on high-value targets on Linux (Ubuntu)	7
Fig.10 Windows Audit Verification (Failed Logins)	8
Fig.11 Windows Audit Verification (Folder/File Creation and Access)	8
Fig.12 Linux (Ubuntu) Log File showing triggered events	9

1. Problem Statement

In an evolving threat landscape, the absence of robust auditing protocols creates a blind spot for unauthorized access and internal data tampering. Without a granular log of failed logins or file manipulations, an organization lacks the forensic evidence required to respond to breaches or maintain regulatory compliance. This project addresses the critical need for a transparent, verifiable audit trail across our hybrid Windows and Linux infrastructure to ensure all security events are captured, documented, and ready for forensic analysis.

2. Introduction

This summary details the successful configuration of security auditing events on both a Windows Server environment and an Ubuntu Linux instance hosted on Proxmox. The objective was to implement proactive monitoring for failed authentication attempts and unauthorized object access as part of a comprehensive security strategy. Leveraging Windows Group Policy and the Linux Audit Daemon (auditd) allowed for the establishment of a centralized verification method to ensure system integrity and accountability across the enterprise.

3. Methodology: Steps Taken

To maintain a rigorous security posture, the transition from passive monitoring to active oversight begins with a comprehensive Audit Policy Setup. The Local Group Policy Editor (gpedit.msc) was used to define a cross-section of 17 distinct audit events, ranging from account logons to privilege use, ensuring that administrators can capture every critical interaction within the OS for future review. This foundational layer serves as the eyes of the system, recording the granular data points necessary for deep forensic analysis.

However, configuration alone is insufficient without robust GPO Enforcement to guarantee that these policies remain active and uncompromised. Enabling the *Force audit policy subcategory settings* within *Security Options* helps the system ensure that advanced, specific configurations override basic category-level rules, preventing technical conflicts that could lead to blind spots in the logs. This enforcement layer acts as the backbone of the auditing framework, ensuring that high-level security mandates are strictly applied at the kernel level rather than being suppressed by legacy settings.

Once these global policies are strictly enforced, the technical focus shifts to granular *Object Auditing* to protect the organization's most sensitive digital assets. This was implemented by creating a dedicated directory at *C:\AuditTest* and configuring advanced auditing properties for the *Everyone* principal, specifically to track file and folder creation. Utilizing *inheritance* to track every document created or modified within these directories helped create a transparent trail of accountability for all file-level manipulations. Together, these three pillars form a cohesive defense strategy that successfully transforms raw system data into actionable security intelligence.

3.1 Windows Server Configuration

- **Policy Definition:** The *gpedit.msc* utility was used to select a diverse cross-section of 17 events to ensure broad visibility into system behavior. In addition to *Audit Object Access* and *Audit Logon Events*, below is a cross-section of 15 other Events enabled for audit purpose:

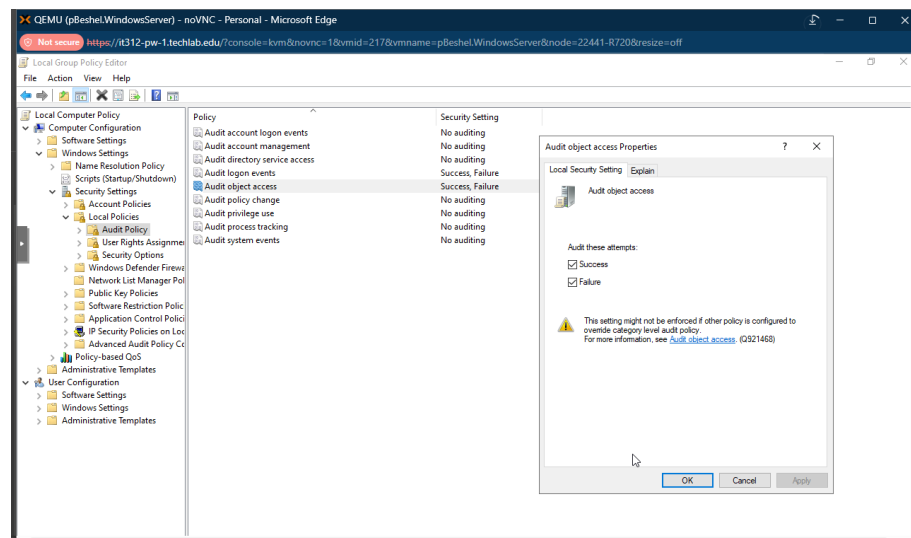


Fig.1 Object Access and Logon Events Audit policy setup

- **Folder: Account Logon**
 - i. **Audit Credential Validation:** Essential for tracking all login attempts.

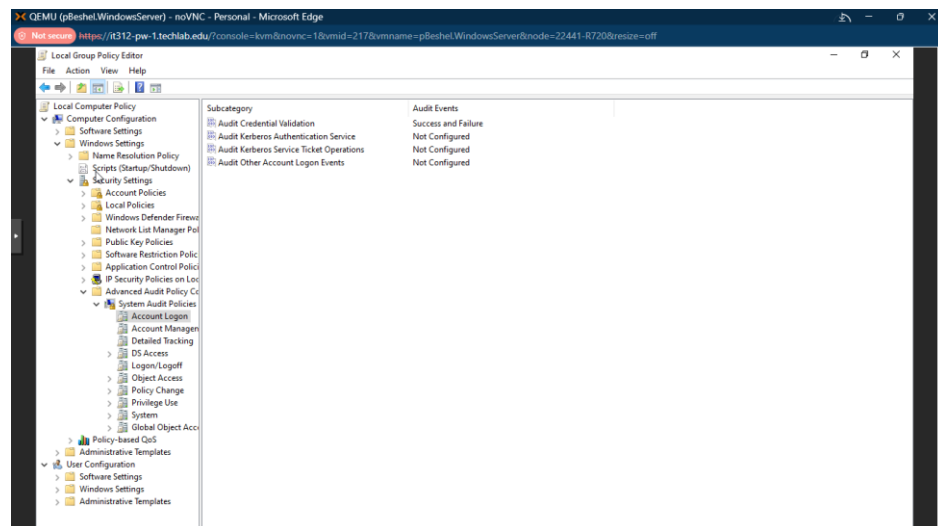


Fig.2 Audit Credential Validation

- **Folder: Account Management**
 - ii. **Audit User Account Management:** Logs when users are created or deleted.
 - iii. **Audit Security Group Management:** Logs changes to high-level access groups.

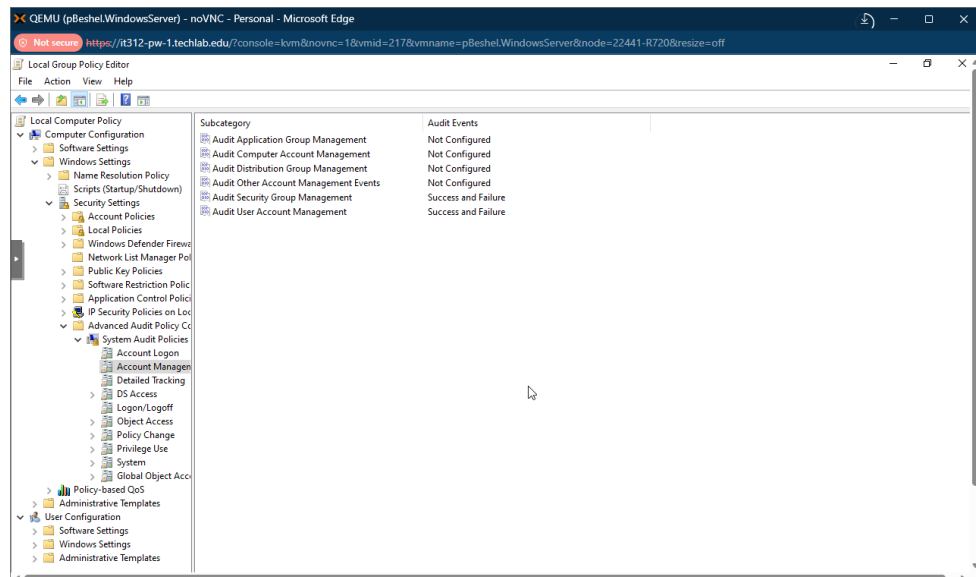


Fig.3 Audit User Account Management and Audit Security Group Management

- **Folder: Logon/Logoff**

- iv. **Audit Account Lockout:** Tracks when a user is locked out due to too many wrong passwords.
- v. **Audit Logoff:** Tracks when users leave the system.
- vi. **Audit Special Logon:** Logs when someone logs in with administrative privileges.

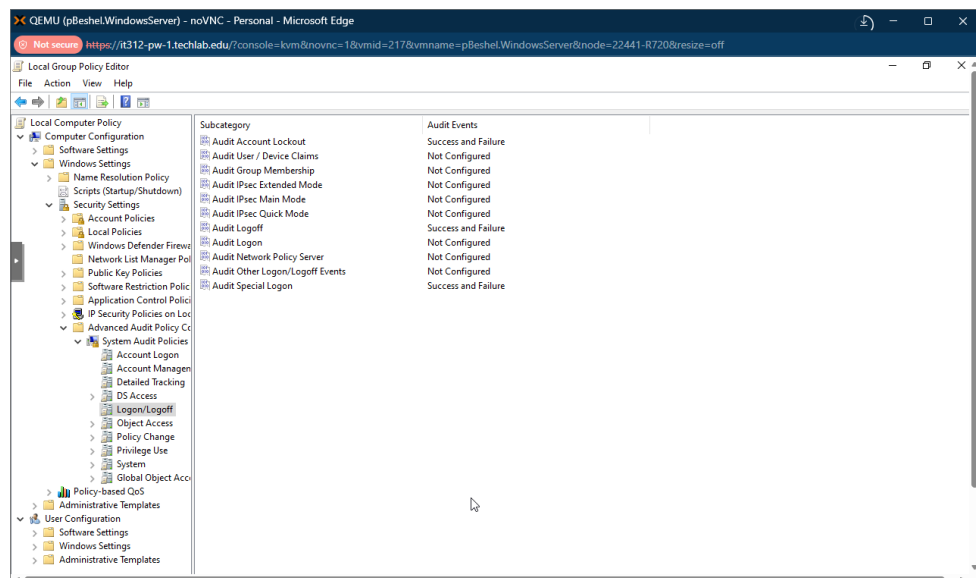


Fig.4 Audit Account Lockout, Audit Logoff, and Audit Special Logon

- **Folder: Object Access**

- vii. **Audit File System:** This is the detailed version of your file creation audit.
- viii. **Audit Registry:** Logs changes to system settings.

ix. **Audit Kernel Object:** High-level system security events.

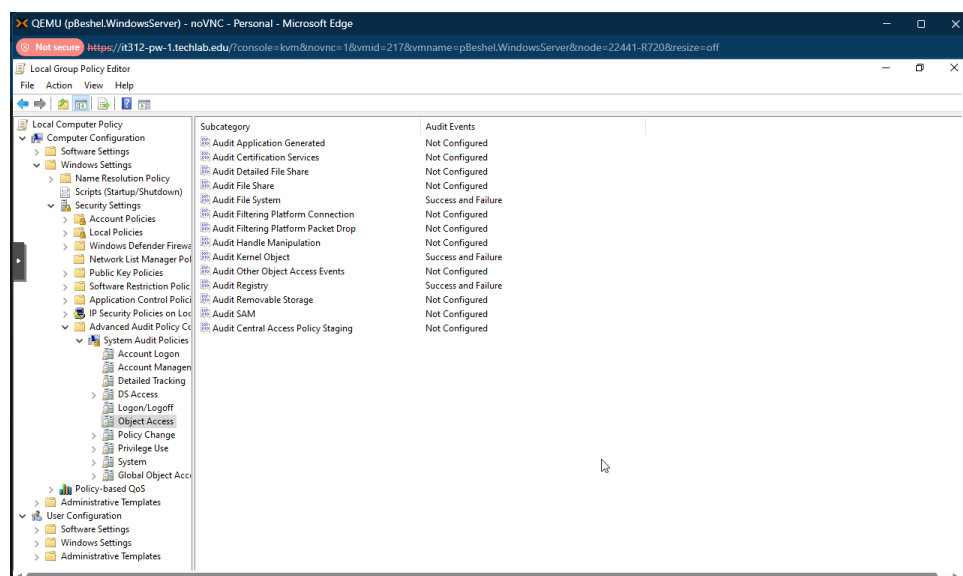


Fig.5 Audit File System, Audit Registry, and Audit Kernel Object

- **Folder: Policy Change**

- x. **Audit Authentication Policy Change:** Logs if someone tries to weaken password requirements.
- xi. **Audit Authorization Policy Change:** Logs if someone changes who is allowed to do what.

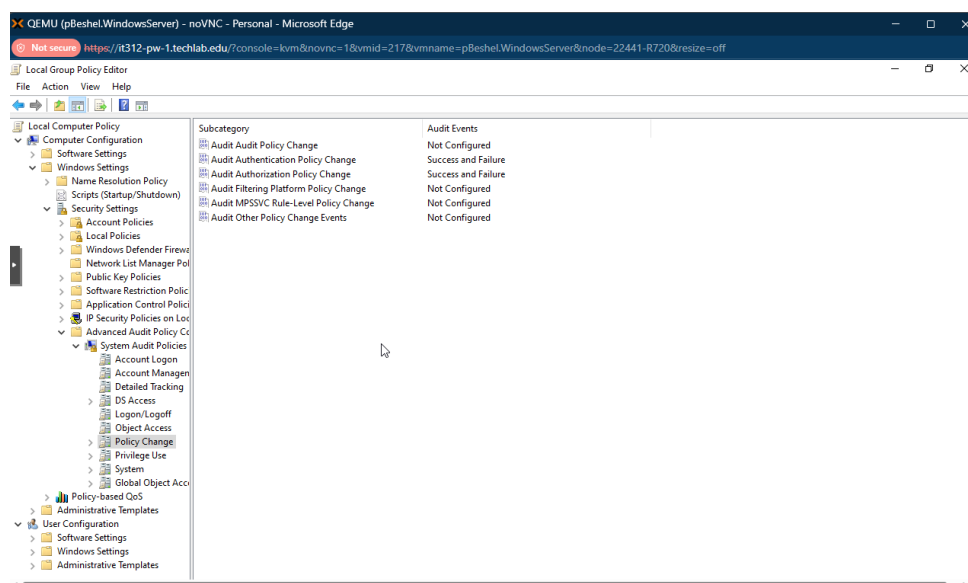


Fig.6 Audit Authentication Policy Change and Audit Authorization Policy Change

- **Folder: Privilege Use**

- xii. **Audit Sensitive Privilege Use:** Logs powerful actions (like taking ownership of a file).
- xiii. **Audit Non-Sensitive Privilege Use:** Logs minor actions (like changing the system time).

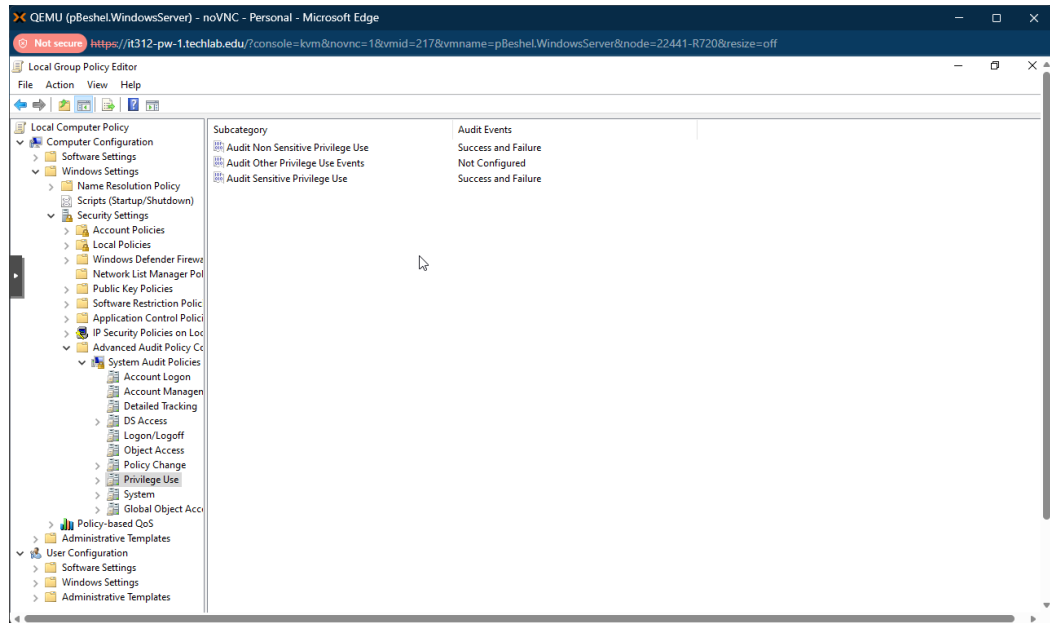


Fig.7 Audit IPsec Driver, Audit Security State Change, and Audit System Integrity

- **Folder: System**

- xiv. **Audit IPsec Driver:** Security for network traffic.
- xv. **Audit Security State Change:** Logs when the auditing system itself starts or stops.
- xvi. **Audit System Integrity:** Logs attempts to violate the system's security.

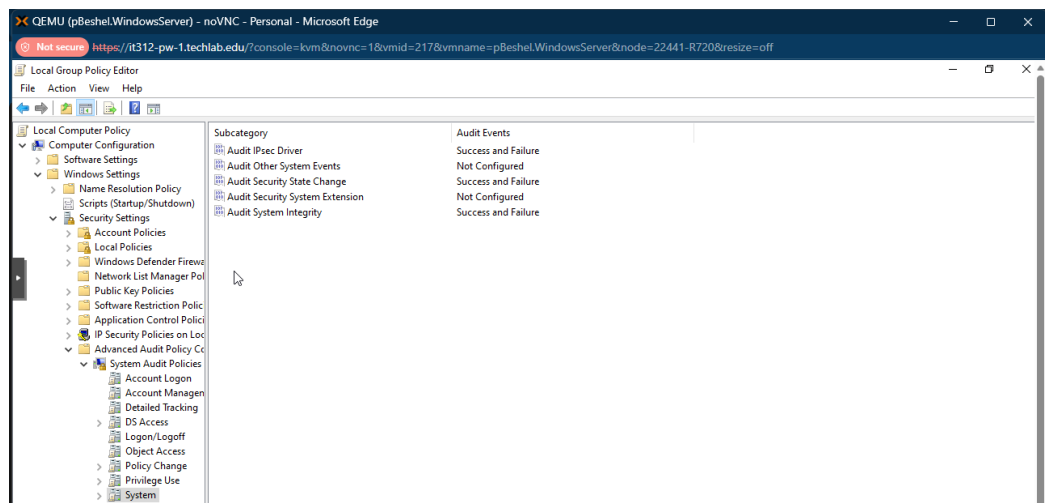
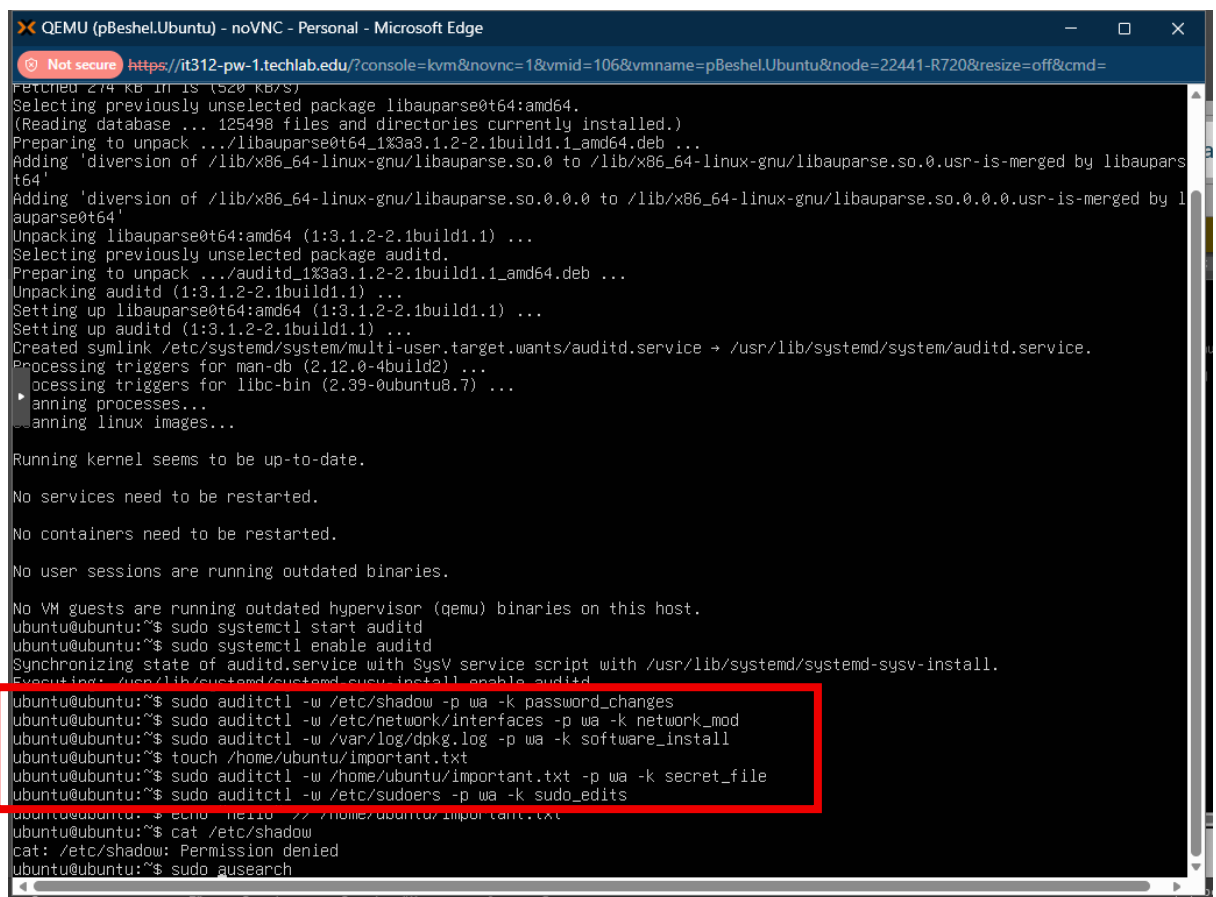


Fig.8 Audit IPsec Driver, Audit Security State Change, and Audit System Integrity

- ✓ **Enforcement Toggle:** The override setting was toggled to *Enabled* to ensure subcategory specificity was maintained.
- ✓ **Targeted Monitoring:** The *C:\AuditTest* environment was established as the primary zone for testing object access and inheritance logic.

3.2 Linux (Ubuntu) Auditing

- **Service Deployment:** The *auditd* service was installed and enabled on the Ubuntu VM via Proxmox to monitor kernel-level system calls.
- **Rule Logic:** Using *auditctl*, five specific watches were placed on high-value targets like */etc/shadow* and */etc/sudoers* to detect unauthorized modifications.



```

QEMU (pBeshel.Ubuntu) - noVNC - Personal - Microsoft Edge
https://it312-pw-1.techlab.edu/?console=kvm&novnc=1&vmid=106&vmname=pBeshel.Ubuntu&node=22441-R720&resize=off&cmd=
fetching 274 KB in 1s (520 KB/s)
Selecting previously unselected package libauparse0t64:amd64.
(Reading database ... 125498 files and directories currently installed.)
Preparing to unpack .../libauparse0t64_1%3a3.1.2-2.1build1.1_amd64.deb ...
Adding 'diversion of /lib/x86_64-linux-gnu/libauparse.so.0 to /lib/x86_64-linux-gnu/libauparse.so.0.usr-is-merged by libaupars
t64'
Adding 'diversion of /lib/x86_64-linux-gnu/libauparse.so.0.0.0 to /lib/x86_64-linux-gnu/libauparse.so.0.0.0.usr-is-merged by l
auparse0t64'
Unpacking libauparse0t64:amd64 (1:3.1.2-2.1build1.1) ...
Selecting previously unselected package auditd.
Preparing to unpack .../auditd_1%3a3.1.2-2.1build1.1_amd64.deb ...
Unpacking auditd (1:3.1.2-2.1build1.1) ...
Setting up libauparse0t64:amd64 (1:3.1.2-2.1build1.1) ...
Setting up auditd (1:3.1.2-2.1build1.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/auditd.service → /usr/lib/systemd/system/auditd.service.
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Planning processes...
Planning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ubuntu:~$ sudo systemctl start auditd
ubuntu@ubuntu:~$ sudo systemctl enable auditd
Synchronizing state of auditd.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable auditd
ubuntu@ubuntu:~$ sudo auditctl -w /etc/shadow -p wa -k password_changes
ubuntu@ubuntu:~$ sudo auditctl -w /etc/network/interfaces -p wa -k network_mod
ubuntu@ubuntu:~$ sudo auditctl -w /var/log/dpkg.log -p wa -k software_install
ubuntu@ubuntu:~$ touch /home/ubuntu/important.txt
ubuntu@ubuntu:~$ sudo auditctl -w /home/ubuntu/important.txt -p wa -k secret_file
ubuntu@ubuntu:~$ sudo auditctl -w /etc/sudoers -p wa -k sudo_edits
ubuntu@ubuntu:~$ echo "Hello" >> /home/ubuntu/important.txt
ubuntu@ubuntu:~$ cat /etc/shadow
cat: /etc/shadow: Permission denied
ubuntu@ubuntu:~$ sudo ausearch

```

Fig.9 Five specific watches set on high-value targets on Linux (Ubuntu)

4. Results Obtained

The auditing system was stress-tested to ensure that events were being correctly captured in the respective security logs.

4.1 Windows Audit Verification

- **Failed Logins:** Generated multiple failed login attempts; Windows successfully recorded these as *Event ID 4625* (Audit Failure) in the Security Log.

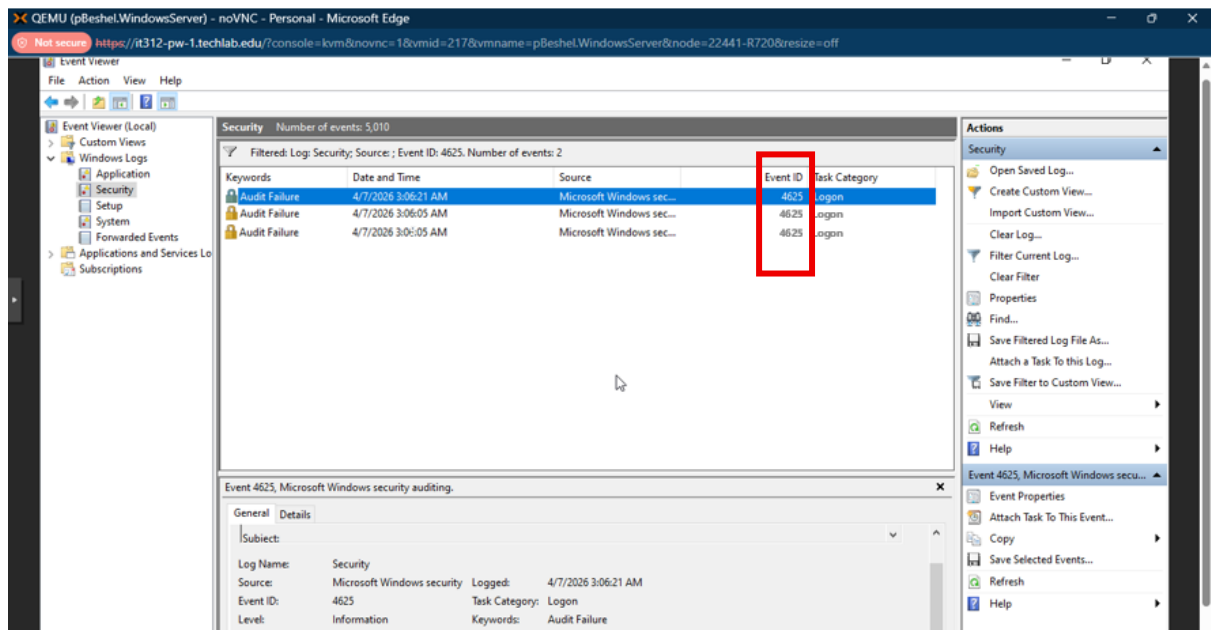


Fig.10 Windows Audit Verification (Failed Logins)

- **Object Access:** Successfully logged the creation of 3 folders and 9 documents. Enabling inheritance, the system correctly identified *Event ID 4663* for child objects within *C:\AuditTest*.

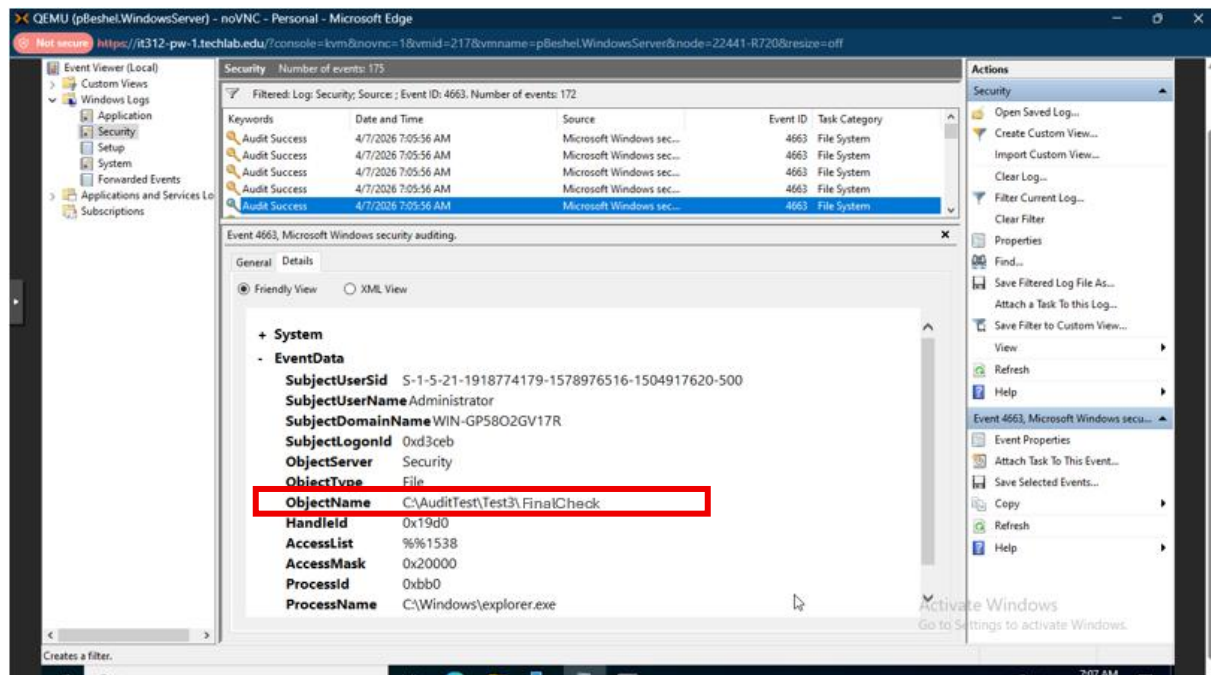
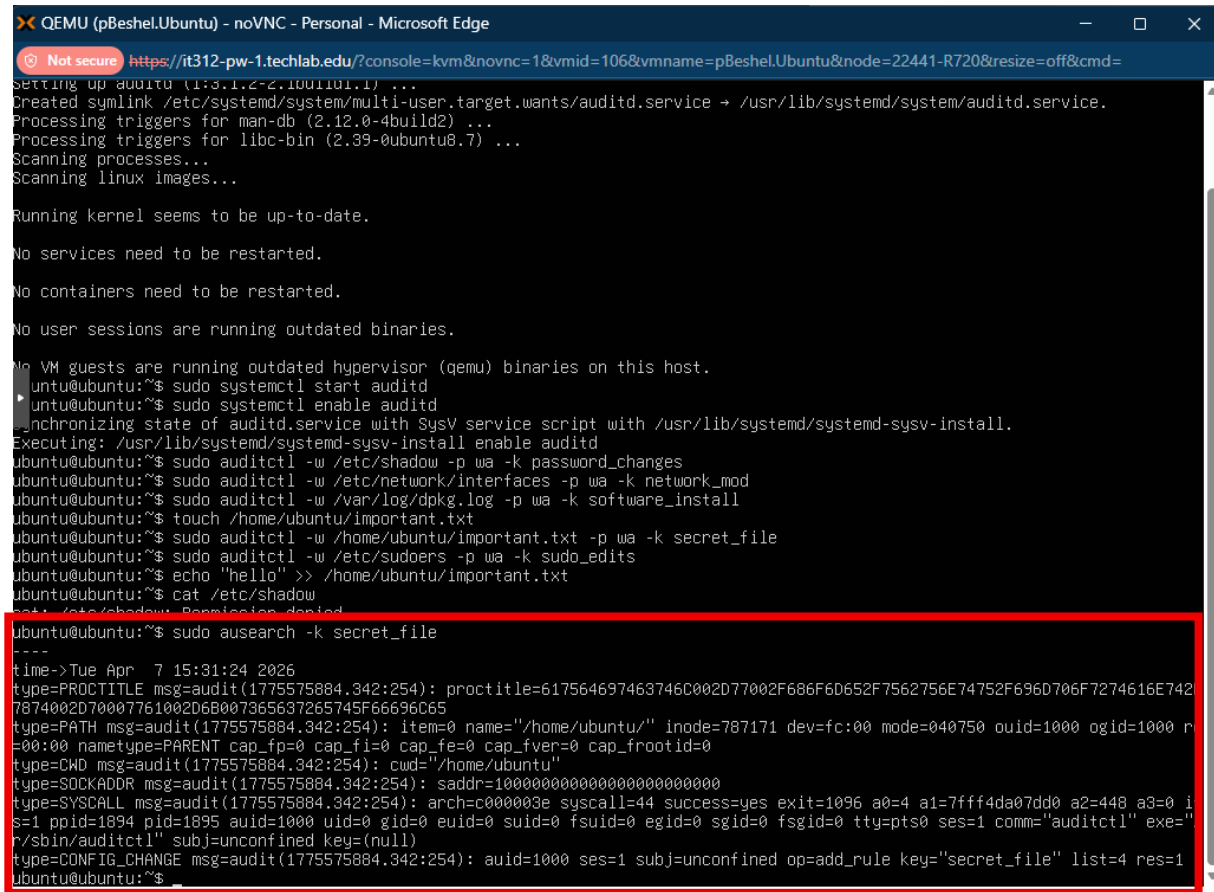


Fig.11 Windows Audit Verification (Folder/File Creation and Access)

4.2 Linux Audit Verification

- **Event Generation:** Triggered multiple events by attempting to read restricted files and modifying watched documents.
- **Log Retrieval:** Used *ausearch* to pull metadata confirming the timestamp, *syscall* type, and the specific user ID (AUID) responsible for the action.



```
QEMU (pBesheL.Ubuntu) - noVNC - Personal - Microsoft Edge
https://it312-pw-1.techlab.edu/?console=kvm&novnc=1&vmid=106&vmname=pBesheL.Ubuntu&node=22441-R720&resize=off&cmd=
Setting up auditd (1:3.1.2-2.1001101.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/auditd.service → /usr/lib/systemd/system/auditd.service.
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ubuntu:~$ sudo systemctl start auditd
ubuntu@ubuntu:~$ sudo systemctl enable auditd
Synchronizing state of auditd.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable auditd
ubuntu@ubuntu:~$ sudo auditctl -w /etc/shadow -p wa -k password_changes
ubuntu@ubuntu:~$ sudo auditctl -w /etc/network/interfaces -p wa -k network_mod
ubuntu@ubuntu:~$ sudo auditctl -w /var/log/dpkg.log -p wa -k software_install
ubuntu@ubuntu:~$ touch /home/ubuntu/important.txt
ubuntu@ubuntu:~$ sudo auditctl -w /home/ubuntu/important.txt -p wa -k secret_file
ubuntu@ubuntu:~$ sudo auditctl -w /etc/sudoers -p wa -k sudo_edits
ubuntu@ubuntu:~$ echo "hello" >> /home/ubuntu/important.txt
ubuntu@ubuntu:~$ cat /etc/shadow
cat: /etc/shadow: Permission denied
ubuntu@ubuntu:~$ sudo ausearch -k secret_file
----
time->Tue Apr  7 15:31:24 2026
type=PROCTITLE msg=audit(1775575884.342:254): proctitle=617564697463746C002D77002F686F6D6552F7562756E74752F696D706F7274616E742
7874002D70007761002D68007365637265745F666696C65
type=PATH msg=audit(1775575884.342:254): item=0 name="/home/ubuntu/" inode=787171 dev=fc:00 mode=040750 ouid=1000 ogid=1000 r
=00:00 nametype=PARENT cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=0
type=CWD msg=audit(1775575884.342:254): cwd="/home/ubuntu/"
type=SOCKADDR msg=audit(1775575884.342:254): saddr=10000000000000000000000000000000
type=SYSCALL msg=audit(1775575884.342:254): arch=c000003e syscall=44 success=yes exit=1096 a0=4 a1=7fff4da07dd0 a2=448 a3=0 i
s=1 ppid=1894 pid=1895 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts0 ses=1 comm="auditctl" exe="
r/sbin/auditctl" subj=unconfined key=(null)
type=CONFIG_CHANGE msg=audit(1775575884.342:254): auid=1000 ses=1 subj=unconfined op=add_rule key="secret_file" list=4 res=1
ubuntu@ubuntu:~$
```

Fig.12 Linux (Ubuntu) Log File showing triggered events

5. Challenges Encountered

- **Event Throttling:** During Windows testing, repeated failed logins did not immediately generate unique entries. This was resolved by implementing a 30-second delay between attempts to bypass system throttling.
- **Inheritance Gaps:** Initially, file-level audits were missing despite folder-level auditing being active. This required a manual override in the Advanced Security settings to push inheritable auditing entries to all child objects.

- **Log Visibility:** Filtering through over 5,000 active security events proved difficult; I utilized Event Viewer's *Filter Current Log* feature to isolate specific Event IDs (4625 and 4663) for analysis.

6. Lessons Learned

- **Log Placement:** Confirmed that login and object audits are stored in the *Security Log*, while system-level events appear in the *System Log*, and third-party software events reside in the *Application Log*.
- **Proactive Defense:** Auditing serves as a proactive defense mechanism to identify brute-force patterns or unauthorized internal reconnaissance before a breach occurs.
- **Complexity of Windows GPO:** Discovered that *Advanced Audit Policy* settings require an explicit override toggle in Security Options to function effectively if basic audit policies are present.

7. Conclusion

The implementation of a comprehensive auditing framework has significantly enhanced the security posture of our Windows and Linux environments. Successfully capturing failed login attempts and unauthorized file modifications helped transition from a reactive state to a proactive monitoring stance. This verifiable audit trail not only ensures accountability but also provides the necessary data foundation for future automated threat detection and incident response protocols.

8. Recommendations

- **Log Management:** Implement a log rotation policy to prevent the Security Log from reaching its maximum size, which would stop new events from being recorded.
- **Automated Alerting:** Integrate these auditing events with a notification system to provide real-time alerts when high-severity events exceed a specific threshold.